Introduction to `batonutil`: A Platform for Agency Pool-level Prepayment Modeling

Cello Research

2017-10-25

Introduction

The life-cycle of a mortgage can be conceptualized as consisting of various **payment states** (**Current**, **30-days Delinquent**, **60-Days Delinquent** etc) with various possible paths to get from one state to another. **Loan-level** prepayment models typically explicitly model the transition between different payment states as a function of various explanatory variables ranging from macro-economic time series to borrower-level attributes. A **pool-level** model can be regarded as a simplified loan-level model that attempts to predict the behavior of mortgage loans at an aggregate level, i.e., by grouping them. Grouping or pooling loans means that we no longer have the ability to track changes in payment status except for *terminal* ones – the transition from "Current" to "Prepayment" or the transition from "Deliquent" to "Prepayment". The upshot is that pool-level models are specified and estimated somewhat differently than loan-level models.

The `batonutil` R package aims to create a one-stop platform for specifying, estimating and tracking an agency pool-level prepayment model. It follows a data science paradigm (Wickham and Grolemund 2017) by breaking down the modeling workflow as per the following figure:

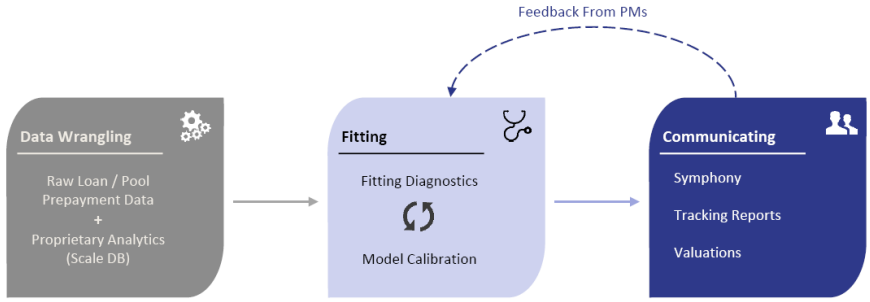

Figure 1: The Prepayment Modeling Workflow

Let's go through these blocks one by one:

1. **Data Wrangling**: This involves getting the "raw" prepayment data set into a useful form for visualization and modeling. There are three parts to this:

   - **Import**: This reads in data from a flat file (typically a .csv) into R. The key function is `imprtr`.
   - **Tidy**: This is a consistent way of storing data that makes it easier to transform, visualize and model.[1] It also handles missing values. The key function is `tidyr`.
   - **Transform**: This function creates new variables in order to make the data a little easier to work with or to perhaps add additional model covariates that can be expressed as functions of some of the inputs. The key function is `tfrmr`.

2. **Fitting**: A model is specified and estimated in this step. Often, several iterations can be involved as specifications are tweaked and parameters are assessed for their reasonableness. The key functions here are `estimatr` and `trackr`.

3. **Communicating**: The model and its tracking results are deployed and made available through the Symphony application to Portfolio Managers (PMs). By assessing model tracking results for individual bonds and through generating hedge ratios, Durations and Convexities for benchmark liquid MBS such as TBAs and IOs, PMs communicate problem areas for the current prepayment model.

The idea behind "institutionalizing" this workflow is to enforce a consistent framework for our various prepayment modeling efforts. This consistency makes it easier to organize, share, check errors, and innovate. The workflow is captured in `main.R`:

```
# Step 0: Set up Global Variables
source("R/config.R")

# Step 1: Import prepayment data into R from flat file
ppdata.imp <- imprtr()

# Step 2: Tidy R data (see source file for definition of 'tidy')
ppdata.td <- tidyr(coll=pkg.env$coll, ppdata=ppdata.imp)

# Step 3: Transform R data (add any new columns if necessary); filter data if necessary
ppdata.tf <- tfrmr(coll=pkg.env$coll, submodel=pkg.env$submodel,
                   gnmaProgram=pkg.env$gnmaProgram, ppdata=ppdata.td)

# Step 4: Estimate submodel
# This step is optional. One can go directly to Step 5 for a more manual,
# iterative & visual fitting process.

# 4.1: Find variables corresponding to best linear model
bestModelsInfo <- selectr(coll=pkg.env$coll, ppdata=ppdata.tf)
cat("Best variables for model: ", bestModels$bestVariables, "\n")

# 4.2: Automated estimation of spline-based model
mdl <- estimatr(coll=pkg.env$coll, submodel=pkg.env$submodel,
                gnmaProgram=pkg.env$gnmaProgram,
                extparamsFitted = pkg.env$extparamsFitted, ppdata = ppdata.tf)
ppdata <- mdl$ppdata

# Step 5: Tracking
# Use filters to track on specific subsets of the data
ppdata.sub <- ppdata.tf %>% dplyr::filter(   (ppdata.tf$bal >  pkg.env$minBal))
```

[1] See http://r4ds.had.co.nz/tidy-data.html.

```
ppdata.fin <- trackr(coll = pkg.env$coll, submodel = pkg.env$submodel,
                     gnmaProgram = pkg.env$gnmaProgram, ppdata = ppdata.sub,
                     bucketSize = pkg.env$bucketSize)
```

In practice, the initial csv file is produced using a SQL query on a loan/pool database. "Tidying" the data turns out to be more convenient in SQL (or Python) for example in tasks such as filling in NA values with an imputation algorithm for millions of rows. Consequently, the `tidyr` is currently more or less a placeholder to handle legacy data sets that have not been fully "tidied" before being imported into R.

References

Wickham, Hadley, and Garrett Grolemund. 2017. *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. Book. http://r4ds.had.co.nz/.